Wildfire Evacuation Planner: Max Flow and Min Cut on Los Angeles Roads

Course: COMP 482 (Algorithms), CSUN. Fall 2026. Instructor: Misbah Hoque

Team: Yevhen Mishchenko (Implementation Lead), May Thu Soe Moe (Project Lead), Abigail Artiga (Algorithms Lead), Juan Razo (Experiments Lead), Nicole Uribe(Report and Presentation Lead)

1. Introduction (Nicole)

When the Palisades fire hit in January 2025, mandatory evacuation orders covered about 30,000 people, and the roads could not handle it. Sunset Boulevard and Pacific Coast Highway were gridlocked, and a bulldozer had to clear abandoned cars aside (Fox LA). That made us wonder and ask ourselves a simple question, how many cars per hour can the roads in Pacific Palisades actually move to shelters, and which road is holding everything back?

We answer it with max flow and min cut on the real road network of Pacific Palisades. We write the algorithms ourselves, prove the algorithms are correct, and check them against brute force on small graphs with timing experiments on real and synthetic networks. We also ask ourselves two what-ifs: which road closure costs the most, and which single road is the best to widen.

Related work: Borgwardt et al. (2024) model wildfire evacuation as max flow on a time-expanded network. We implement and compare the algorithms ourselves, starting from the static problem.

2. Problem Statement (Abigail)

The problem we are trying to solve is how many cars per hour the roads of the Pacific Palisades can move from homes to shelters and which roads will slow people down during an evacuation. We will be treating the road map as a directed graph where each intersection represents a node, and each direction of a road is an edge with a capacity in cars per hour. Each capacity is a whole number, found from the number of lanes and the type of road. Shelters are placed where the roads leave the area, since the goal of an evacuation is to get people out. All homes are connected to one starting point s and all shelters to one ending point t, and then we find the maximum flow from s to t of the most cars per hour the roads can carry. The edges from s and to t are given a capacity so large that they never limit the answer, so only real roads can be bottlenecks. We will also find the minimum cut of the roads that limit that flow, which are our bottlenecks. If every road in the cut were closed, no car could reach a shelter, and the max flow always equals the total capacity of the smallest such cut. Then for each road we will check how much flow is lost if it is closed and how much is gained if there is an added lane. This tells us which closures would hurt the most and which single road is the best one to widen.

3. Algorithms (Abigail)

For the algorithm implementation, we will use and compare Ford-Fulkerson and Edmonds-Karp because they both find the maximum flow. These algorithms repeatedly find a path from s to t with spare capacity and push as many cars along it as possible, then stops when no such path is available. The Ford-Fulkerson algorithm finds paths with depth-first search (DFS) and runs in $O(E \cdot f^*)$ time, where f^* is the max flow value. Since every capacity is a whole number, each path adds at least one car per hour, so Ford-Fulkerson always finishes. As for Edmonds-Karp, it will always pick the shortest path using breadth-first search (BFS) and runs in $O(VE^2)$ time. Because Ford-Fulkerson's time depends on the flow value, it may still be faster than Edmonds-Karp on our roads if the flow is small, and experiment E1 will test this. After they are complete the intersections that are still reachable from s give the minimum cut in $O(V+E)$ time. The roads leaving that set of intersections are the bottleneck roads. The max flow and min cut theorem states that the two are equal should be correct. Then on small graphs, a brute force check of every possible cut will help confirm our results. Brute force is too slow for the real map, since the number of possible cuts doubles with every intersection, but it is simple enough that we can trust its answer. We will be rerunning max flow once for each road for our what-ifs to add onto our results.

4. Data (Juan)

Our project will use road network data from the Pacific Palisades, Los Angeles, gathered by OpenStreetMap using the Python library OSMnx. The dataset will contain intersections, roads connected to each other, road type, speed limits, and the number of lanes. Intersections will be represented as nodes, and roads will be represented as edges. Each road will be assigned an estimated capacity measured in cars per hour based on the number of lanes and the Highway Capacity Manual. These values are based on assumptions rather than actual traffic measurements, and so we will mention these limitations in the report. We will also identify possible evacuation shelters and use residential locations as starting points for evacuating cars. US Census Data may help estimate household locations around Pacific Palisades. Our team will discuss the exact household locations and assumptions. In addition to the real road data, we will use a random seed generator to create a synthetic network with different numbers of intersections and roads. Using a fixed seed will allow us to reproduce the same network when we test our algorithm. Once we download the real road network, we will record the actual number of nodes and edges.

5. Evaluation Plan (Juan)

We plan to run three experiments to verify the algorithm works and to assess its efficiency. We will run these tests by using real road data from the Pacific Palisades and randomly generated networks. In the first experiment, we will compare the Ford-Fulkerson and Edmonds-Karp algorithms to see which is best. The goal is to use the same road network for the algorithms and

measure the execution time, the number of paths, and max-flow results. We expect the Edmonds-Karp algorithm to stay within its time complexity O(VE^2), while Ford-Fulkerson depends on the flow value. For the second experiment, we will check what will happen when a road closes. First, we will calculate the max flow when all roads are open. Then, we will close roads, recalculate the max flow with the closed roads, and check the capacity lost. Closing a min-cut road will decrease the maximum flow. However, road closures besides the min-cut road can also decrease the max flow. Lastly, for the third experiment, we will increase the capacity by widening the min-cut road. We will compare widening a min-cut road with other regular roads to see which delivers the best outcome. We expect that widening a min-cut road does not always increase road capacity. To check correctness and performance, we will use brute-force on small graphs, NetworkX, repeat experiments multiple times, and use fixed random seeds. We will record median execution time and create a log-log graph to check algorithm performance as network size increases.

6. Implementation (Yevhen)

We will implement every algorithm in Python 3.11 or newer, with our own code rather than library calls. Each module will follow the pseudocode in our final report, so the runtime analysis describes the code we actually run. We will check for correctness in two ways. First, each algorithm will be compared against a brute-force solver on thousands of small random graphs. Second, it will be compared against the max-flow function of the NetworkX library, which we use only in tests and never in our own algorithms. The algorithms will also count their own operations, such as augmenting paths found and edges examined, so the experiments can test the Big-O bounds directly, not only measure running time.

Road data will come from OpenStreetMap through the OSMnx library, and plots will be made with matplotlib. Every experiment will run with one command listed in the repository's README, use fixed random seeds, and write its tables and plots to a results folder, so every number in our report can be regenerated. Automated tests and style checks already run on every change to the repository.

Repository: https://github.com/Draganoider/COMP-482-Project

7. Stretch Goals (Nicole)

If time allows, the static model will be extended in three ways:

- Fire extension: a fire spreads from an ignition point and closes roads over time (Dijkstra over map cells, with wind). Max flow on a time-expanded network then shows how many cars reach a shelter before their roads close (experiment E4).
- Shortest-route comparison: everyone drives the shortest route to get to the nearest shelter (Dijkstra on the roads), and we compare that against the max-flow plan (experiment E5).
- Map app for the presentation: start a fire, watch the cars, see where the bottlenecks are.

8. Timeline and roles (May)

As the Project Lead, I will coordinate with team members to make sure each milestone is completed on time. Each member is responsible for their assigned tasks, and we will communicate regularly to share progress, discuss challenges, and help each other when needed. I will also keep track of deadlines, check the overall project progress, and help ensure that all sections are ready before the final submission.

Milestone	Owner	Done by
Proposal submitted	May Thu Soe Moe	10/31/2026(deadline)
Road data and capacities	Juan Razo	10/30/2026
Problem statement and proofs drafted	Abigail Artiga	10/30/2026
Max flow, min cut, what-ifs, with tests	Yevhen Mishchenko	10/30/2026
Experiments E1–E3 recorded	Juan Razo	10/30/2026
Stretch goals, if time allows	Everyone	10/30/2026
Report and slides	Nicole Uribe	10/30/2026

References (Nicole)

- Kleinberg and Tardos, *Algorithm Design* (chapter 7)
- Borgwardt et al., *Evacuation Planning on Time-Expanded Networks with Integrated Wildfire Information*, arXiv 2410.14500
- OpenStreetMap and OSMnx; the Highway Capacity Manual
- Fox LA report on the Palisades fire abandoned cars